QuadFormer : Representation-Enriched Transformer for TSP via Recursive Fuzzy **Qua**drisection and **D**ensity Token Embedding

Jieun Yook[1], Junpyo Seo[2], Byung-Ro Moon [*]

[1,2,*]Department of Computer Science, Seoul National University, Gwanakro 1, Seoul, 08826, Seoul, South Korea.

*Corresponding author(s). E-mail(s): moon@snu.ac.kr;
Contributing authors: yookje@snu.ac.kr; jpseo99@snu.ac.kr;

Abstract

We propose QuadFormer, a Transformer-based solver for the Traveling Salesman Problem (TSP). Acknowledging the limited informational scope of embeddings in prior Transformer-based TSP solvers, QuadFormer aims to enhance the model's representational capacity through a more geometrically enriched embedding scheme. To this end, we introduce the following key methodological innovations. First, Recursive Fuzzy Quadrisection generates spatial representations by hierarchically encoding nodes across multiple holographic resolutions. Second, density embeddings capture geographic node distributions, employing both directionless and directional forms to represent overall crowdedness and direction-specific concentrations. Third, a Context-Binding Matrix is inserted after the decoder's final layer to facilitate smooth information transfer between encoder and decoder. This matrix overcomes the weak token identity specific to TSP, where node representations lack consistent semantic meaning, unlike tokens in language models. By processing the encoder output, it strengthens encoder-decoder coupling. Furthermore, to fully exploit these advanced embeddings, Open-Hypothesis Decoding is applied during inference, branching multiple candidates only at the initial step to mitigate premature decision risks. Combined, these methods allow QuadFormer to effectively explore spatial distributions of TSP nodes, demonstrating superior empirical performance. The code and data used in the experiments are available at https://github.com/yookje/QuadFormer.git.

Keywords: Neural combinatorial optimization, Traveling salesman problem, Transformer

1 Introduction

The Traveling Salesman Problem (TSP) is a classic combinatorial optimization problem focused on identifying the shortest Hamiltonian cycle connecting a given set of cities or nodes. As a well-known NP-Hard problem, it serves as a fundamental prototype for numerous real-world optimization challenges, where computational costs for exact solutions increase exponentially with problem size. Various TSP-related optimization problems, such as semiconductor VLSI design optimization and vehicle routing problems, frequently arise in industrial applications.

The Transformer model, which has achieved remarkable success in natural language processing, vision, and generative tasks, is now being extended to the optimization domain. In recent years, Transformer-based models have demonstrated the potential to rapidly identify high-quality approximate solutions for large-scale instances. However, their performance still falls short of the exceptional efficacy demonstrated by mature traditional heuristics and search methods—such as memetic genetic algorithms (Merz et al., 2001; Honda et al., 2013; Seo and Moon, 2002) and branch-and-bound (Applegate et al., 1998). As demonstrated in the field of computer vision, applying Transformer architectures to the TSP appears to be in a transitional phase toward surpassing classical approximation methods.

The expressiveness of the input representation significantly impacts Transformer performance. The evolution of token embedding can be viewed as a history of endeavors to enrich input representations. In the original Transformer model (Vaswani, 2017), positional encoding was introduced to embed sequential causality when mapping arbitrary tokens into a real-valued vector. BERT (Kenton and Toutanova, 2019) incorporated segment encoding and bidirectional context to capture diverse linguistic nuances. Other studies have enhanced token expressiveness by integrating additional features, such as TF-IDF values for words in a document (Arora et al., 2017). Vision Transformers employ positional embeddings that reflect the 2D relationships between image patches. Mei et al. (2024) increased the flexibility of input representation by using variable-sized superpixels instead of fixed-size patches. ERNIE (Zhang et al., 2019) leveraged external knowledge graphs to augment the input representation of language models. AlphaFold-2 (Jumper et al., 2021) exemplifies the maximization of input expressiveness by constructing a 2D matrix from a multiple sequence alignment of representative homologs and incorporating a residue correlation matrix as additional input.

Previous studies for TSP Transformers (Kool et al., 2018; Kwon et al., 2020; Ma et al., 2021; Jin et al., 2023; Fang et al., 2024) all utilize 2D coordinates explicitly. Kwon et al. (2020) proposed using embedding augmentation via reflection and rotation transformations, while Jin et al. (2023) incorporated the $arctan\frac{y}{x}$ of the coordinates. Nevertheless, these coordinate-based encodings leave room for more effectively capturing the relative positional and structural information of nodes. For instance, as illustrated in Figure 3, relying solely on 2D coordinates has inherent limitations in sufficiently describing complex spatial relationships. This becomes particularly evident as instance size increases: the average distance between vertices decreases, making the relationship between two nodes difficult to characterize based solely on their Euclidean coordinate differences. Our work on token embedding begins with the recognition that

encodings explicitly dependent on 2D coordinates have limited expressiveness. This paper introduces novel spatial representation methods that do not explicitly use the 2D coordinates of TSP instances. These representations are designed to work in conjunction with the augmented vertex techniques from POMO (Kwon et al., 2020) and the $arctan(\frac{y}{x})$ embedding from Pointerformer (Jin et al., 2023), aiming to richly and systematically integrate nodal positional and contextual environmental information to enhance model performance. This approach intends to suggest a new direction for Transformer-based TSP models and aspires to provide a framework applicable to other combinatorial optimization problems involving spatial layout.

This study proposes QuadFormer, a novel embedding methodology tailored for the TSP domain and a Transformer architecture adapted for this embedding. The core contributions of this work are as follows:

- We propose two novel feature representation methods for geographically enriched input embedding. First, we introduce **Recursive Fuzzy Quadrisection (RFQ)**, which represents 2D coordinates not as mere real numbers but from diverse hierarchical perspectives by recursively quadrisectioning the coordinate space. The coordinate plane is recursively divided into quadrants, and the region membership of vertices at each level is one-hot encoded. In addition, a thin fuzzy rectangle is introduced at the quadrant boundaries to group proximate points on either side, supplementing adjacency information. Second, we propose a density embedding to reflect the geographical information around each vertex. Through **Directionless Density Embedding** and **Directional Density Embedding**, we effectively capture both the overall crowdedness around a vertex and the local density in specific directions.
- We propose **Open-Hypothesis Decoding** (OHD), a novel inference strategy designed to mitigate the risk of premature commitment during tour construction. Recognizing that the selection of the first node occurs under maximal uncertainty yet critically determines the final tour quality, OHD simultaneously explores multiple promising paths. The method branches only at the initial step into the top m most probable nodes, completing each path thereafter via conventional greedy selection. By concentrating computational resources at this pivotal decision point rather than throughout the entire sequence, OHD uncovers high-quality solutions often missed by single-path greedy approaches, while maintaining computational efficiency.
- We customized the architecture of the original Transformer, targeted for language domains, to better suit the TSP domain. Specifically, we replace the standard linear matrix following the decoder with a **Context-Bindng Matrix**. This is designed to ensure the encoder output—the encoded graph representation—is fully utilized for tour construction. This matrix is derived by processing the encoder output through a feed-forward network (FFN).

Thanks to these improvements in input embedding and architectural modifications, our proposed QuadFormer can more effectively capture the geographical structure of TSP instances. We empirically demonstrate that the newly proposed elements enhance the model's representational power.

2 Related Work

2.1 Conventional TSP Algorithms

Optimization algorithms for solving the TSP can be broadly categorized into two groups based on whether they guarantee optimality. Exact algorithms, which do provide this guarantee—though not the guarantee of polynomial time—utilize methods such as branch and cut, branch and price, and dynamic programming. Among these, Concorde (Applegate et al., 2007) is recognized as one of the most powerful exact solvers for TSP. Based on the branch and cut framework and incorporating various heuristics, Concorde is capable of finding optimal solutions, though only with some luck, for instances containing up to 85,900 nodes.

Extensive research has long been conducted on alternative approaches that focus on efficiently finding high-quality solutions within a given time frame, albeit without pursuing optimality. These methods include the nearest neighbor algorithm, the 2-Opt algorithm (Croes, 1958), 3-Opt algorithm (Lin, 1965), the Lin-Kernighan-Helsgaun (LKH) algorithm (Helsgaun, 2017), and the Edge Assembly Crossover Genetic Algorithm (GA-EAX) (Honda et al., 2013). Among these, LKH algorithm and the GA-EAX are considered state-of-the-art heuristics.

The core idea of the LK algorithm (Lin and Kernighan, 1973) is to obtain a shorter tour by identifying a sequence of k edge exchanges (k-change moves where k varies) and replacing the existing edges with new ones. Starting from an initial tour, the algorithm removes a starting edge and inserts the most attractive edge connected to its endpoint. This process of removal and insertion of the most attractive edge continues, with the vertices connected by newly inserted edges becoming fixed and unavailable for further selection in the current sequence. This iterative process continues until no further moves are possible, with only the sequence yielding the highest cumulative improvement being accepted and the rest discarded, resulting in a new, improved solution. This entire procedure is repeated using the new solution as the starting point until no further improvements are possible. The enhanced LKH algorithm improves by restricting the choice of candidate edges for insertion to a set pre-selected based on a minimum spanning tree, thereby reducing search time. More recently, the NeuroLKH (Xin et al., 2021), which combines the LKH algorithm with a graph neural network, has also been proposed.

Furthermore, hybrid genetic algorithms represent another significant approach, effectively combining genetic algorithms (GAs) with heuristic local search. GAs are a prominent methodology that applies principles of evolution to problem solving. In this paradigm, each potential solution to a given problem is treated as an individual, and a population of these solutions is collectively evolved to discover increasingly better solutions. This evolution is typically guided by genetic operators such as selection, crossover, mutation, and replacement. A recognized limitation of standard GAs is their relative weakness in fine-tuning around a local optimum. Consequently, hybrid GAs (Seo and Moon, 2002; Merz et al., 2001) which integrate a GA with a local search heuristic, are widely regarded as a natural and powerful strategy for achieving satisfactory performance on complex optimization problems.

Table 1: The vertex augmentation strategy proposed by POMO.

Rotation and Inversion of (x, y)	
(x, y)	(y, x)
$(x, 1-y)$	$(y, 1-x)$
$(1-x, y)$	$(1-y, x)$
$(1-x, 1-y)$	$(1-y, 1-x)$

2.2 Transformer-based TSP Algorithms

Transformer-based approaches for TSP began with initial research Kool et al. (2018) that applied the Pointer Network (Vinyals et al., 2015), and various variant models have since been proposed. POMO (Kwon et al., 2020) improved upon the issue of representational redundancy caused by cyclicity by utilizing multiple trajectories with different starting points within the same TSP instance during both learning and inference. This approach reduced the solution bias associated with a single starting point and established a foundation for stable learning. Additionally, Kwon et al. (2020) effectively incorporated invariance to rotation and reflection by augmenting vertices through rotation and symmetry, as illustrated in Table 1.

Subsequently, DACT (Ma et al., 2021) proposed a model that learns not only Node Feature Embedding (NFE) but also Positional Feature Embedding (PFE). Through its Dual-Aspect Collaborative Attention (DAC-Att) mechanism, DACT deeply integrates the interaction between NFE and PFE. Notably, by using Cyclic Positional Encoding (CPE) to initialize the PFE containing positional information, it guided the model to learn the cyclic structure inherent in TSP solutions. Traditional Transformer positional encodings only reflect sequential (1D) information, making it easy to miss the cyclicity where the start and end points connect. CPE overcomes this limitation by directly reflecting the characteristic of TSP routes, in which the first and last nodes are reconnected. Bresson (Bresson and Laurent, 2021) diverged from previous studies that used the first node of the input as the decoding start; instead, they embedded the starting point into the decoder, allowing the model to learn which node is advantageous to start decoding from. They also employed sinusoidal positional encoding in the decoder. Sym-NCO (Kim et al., 2022) focused on the symmetry of TSP solutions and proposed a loss function that utilizes this property during the model's training process. It leverages the fact that an instance x and its rotated counterpart x' share the same optimal solution, guiding the model to minimize the reward difference between the original and rotated instances. Pointerformer (Jin et al., 2023) introduced a reversible residual network into the encoder to enhance memory efficiency and accelerate inference. To effectively reflect invariance to rotation and reflection, it utilized the augmented points proposed by Kwon et al. (2020) (Table 1) and embedded the angle $(arctan(\frac{y}{x}))$ formed by the line connecting the origin to the point (x, y) and the x-axis. INViT (Fang et al., 2024) designed its architecture to operate robustly across data of diverse distributions and scales. This model posits that distant nodes are likely

to contain information irrelevant to the current node and excludes them, instead utilizing information from neighboring nodes to assess the local state and inform the next node selection. It applies a k-nearest neighbors (k-NN) algorithm, using only information from the k nodes closest to the last visited node. Furthermore, it avoids reliance on a single k value by setting multiple k values to form a Nested View, thereby incorporating information at various levels. For example, INViT-3V uses $k = 15, 35, 50$, concatenating the embeddings of the $15, 35, 50$ nearest nodes from the last visited node for use as keys and values in the decoding process. LEHD (Luo et al., 2023) inverted the heavy-encoder light-decoder design, attempting a novel architecture with a lighter encoder and a deeper decoder. During training, it randomly samples partial sequences from the optimal solution, enabling the model to learn how to construct partial solutions. This method enhances the model's generalization capability during inference, as it learns independently of the training data size. Luo et al. (2025) addressed the issue that existing Neural Combinatorial Optimization (NCO) methods, which rely on the self-attention mechanism with $O(N^2)$ complexity, face memory shortages and excessive computation times for large instances. In Luo et al. (2025), they proposed a Lightweight Cross Attention method. Instead of attention between all nodes, it uses attention through representative nodes (the first and last visited nodes), reducing complexity to $O(2N)$. This significantly improves computational efficiency, though it sacrifices adaptability to smaller instances.

3 Methodology

3.1 Overview of Our Framework

QuadFormer modifies the conventional Transformer architecture to better suit the TSP domain while leveraging the strengths of the encoder's universal context representation and the decoder's sequential generative structure. Each token corresponds to an individual node. However, unlike the look-up table (LUT) in language models, which encompasses all tokens in the corpus, QuadFormer's LUT is dynamically reset for each instance and includes only the nodes belonging to that specific instance. As illustrated in Figure 1, QuadFormer first processes raw node coordinates through a Quad Embedder to construct informative input embeddings $e \in \mathbb{R}^{N \times d_{model}}$ that richly encapsulate geographical information. These embeddings are then enhanced with spatial positional encoding(PE), a method similar to DETR (Carion et al., 2020), to incorporate their 2D spatial relationships. The resulting embeddings are subsequently passed through six encoder blocks to produce a comprehensive context embedding(also referred to as memory or graph context vector) of the input graph instance. Following the philosophy of the standard Transformer design, this context embedding serves as the source of keys and values for the encoder-decoder cross-attention mechanism. A key architectural innovation in QuadFormer is the direct feeding of this context embedding into the decoder input, diverging from the typical Transformer setup. Furthermore, this same vector is utilized to construct a Context-Binding Matrix, which replaces the standard final linear layer. For the decoding process, the decoder's input is formed by summing the encoder's context embedding with a CPE, proposed by Ma et al. (2021), to infuse information about the cyclic nature of a valid TSP tour. This combined input

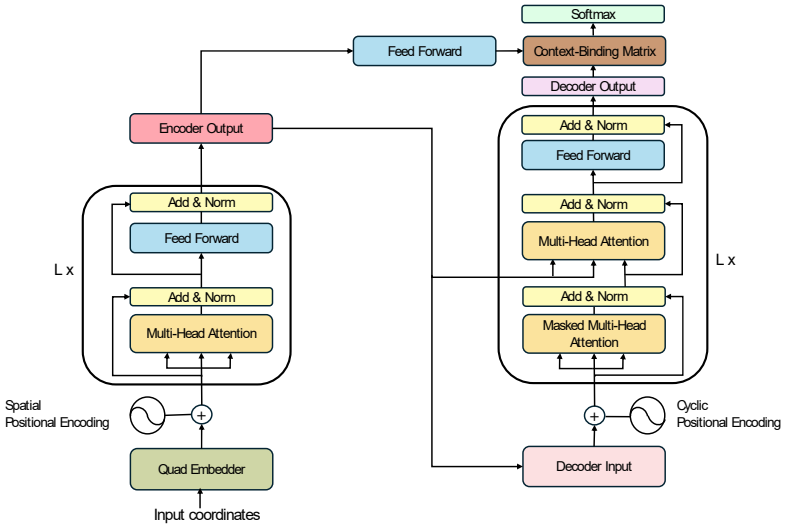


Fig. 1: A framework of **QuadFormer**. The encoder refines Quad embeddings through multiple self-attention layers, producing contextual representations that are further processed by **Context-Binding Matrix**. The decoder then generates the next-node prediction at each step.

is processed by six decoder blocks, transforming it into the final decoder output. A significant departure from the original Transformer occurs at this stage. Instead of projecting the decoder output through a standard learned linear weight matrix followed by softmax, QuadFormer employs a Context-Binding Matrix. The Context-Binding Matrix is the result of abstracting the graph's context embedding (encoder output) through a FFN. The decoder output is multiplied by this Context-Binding Matrix, and a softmax function is applied to the result to produce a probability distribution over the nodes. The next node is selected auto-regressively based on this distribution, with a masking mechanism applied to prevent revisiting already-visited nodes. QuadFormer is trained using supervised learning, with cross-entropy serving as the loss function.

In general, constructive models employ greedy decoding during inference; however, to address the inherent uncertainty associated with early-stage decisions, we propose Open-Hypothesis Decoding. In the TSP, the selection of the first node constitutes a critical decision that significantly influences the entire tour. Nevertheless, at this early stage, the available information is severely limited, making it challenging to ascertain the optimal choice. To resolve this dilemma, OHD performs branching into the top m candidate nodes exclusively at the first step. Each hypothesis is subsequently completed through greedy decoding, and ultimately, the most optimal tour is selected.

It is noteworthy that this paper uses the terms memory, context vector, encoder output and graph context interchangeably to refer to the final output of the encoder.

3.2 Quad Embedder

The core objective of a Transformer is to effectively capture the multi-dimensional relationships between tokens through its training process. Via abstraction through matrix multiplication and the computation of abstract relationships using attention maps, tokens with strong relationships progressively attain a high correlation, while those with weaker relationships attain a low correlation; they are transformed over time. To execute this mechanism effectively, the expressive capacity of the input encoding for spatial representation in graph problems cannot be overemphasized. In QuadFormer, two-dimensional information are processed by the Quad Embedder into initial embeddings endowed with rich spatial expressiveness. These embeddings are subsequently combined with 2D positional encoding to form the input for the encoder. The following section elaborates on our newly proposed methodology for token representation.

Algorithm 1 Quad Embedder

Require: Node coordinates $\mathbf{P} \in \mathbb{R}^{n\times 2}$ and recursion depth k
Ensure: Node embeddings $\mathbf{E} \in \mathbb{R}^{n\times 256}$

$\mathbf{F}_{\text{RFQ}} \leftarrow \textsc{RecursiveFuzzyQuadrisection}(\mathbf{P}, k)$ ▷ $\mathbf{F}_{\text{RFQ}} \in \mathbb{R}^{n\times 8k}$
▷ At each recursion level, a four-dimensional one-hot vector identifies the quadrant containing each node, and an additional four-dimensional binary vector represents its membership in the fuzzy boundary regions.

$\mathbf{F}_{\text{DL}} \leftarrow \textsc{DirectionlessDensity}(\mathbf{P})$ ▷ $\mathbf{F}_{\text{DL}} \in \mathbb{R}^{n\times 8}$
▷ The numbers of neighboring nodes within two circular regions of different radii are discretized and encoded as two four-dimensional one-hot vectors.

$\mathbf{F}_{\text{DR}} \leftarrow \textsc{DirectionalDensity}(\mathbf{P})$ ▷ $\mathbf{F}_{\text{DR}} \in \mathbb{R}^{n\times 48}$
▷ The neighborhood of each node is divided into 12 directional sectors, and the density within each sector is discretized and encoded as a four-dimensional one-hot vector.

$\mathbf{F}_{\text{AUG}} \leftarrow \textsc{EightFoldAugmentation}(\mathbf{P})$ ▷ $\mathbf{F}_{\text{aug}} \in \mathbb{R}^{n\times 16}$
▷ Nodes are augmented through rotations and reflections, resulting in eight variants per original node: (x, y), (y, x), $(x, 1-y)$, $(y, 1-x)$, $(1-x, y)$, $(1-y, x)$, $(1-x, 1-y)$, and $(1-y, 1-x)$.

$\mathbf{F}_{\text{ANG}} \leftarrow \textsc{AngularEmbedding}(\mathbf{F}_{\text{aug}})$ ▷ $\mathbf{F}_{\text{angle}} \in \mathbb{R}^{n\times 8}$
▷ For each augmented coordinate (x, y), the angle between the line connecting the origin to (x, y) and the x-axis is used as an angular feature.

$\mathbf{F} \leftarrow \textsc{Concat}(\mathbf{F}_{\text{RFQ}}, \mathbf{F}_{\text{DL}}, \mathbf{F}_{\text{DR}}, \mathbf{F}_{\text{AUG}}, \mathbf{F}_{\text{ANG}})$ ▷ $\mathbf{F} \in \mathbb{R}^{n\times(8k+80)}$

$\mathbf{E} \leftarrow \textsc{LinearProjection}(\mathbf{F})$ ▷ $\mathbf{E} \in \mathbb{R}^{n\times 256}$

return E

3.2.1 Recursive Fuzzy Quadrisection

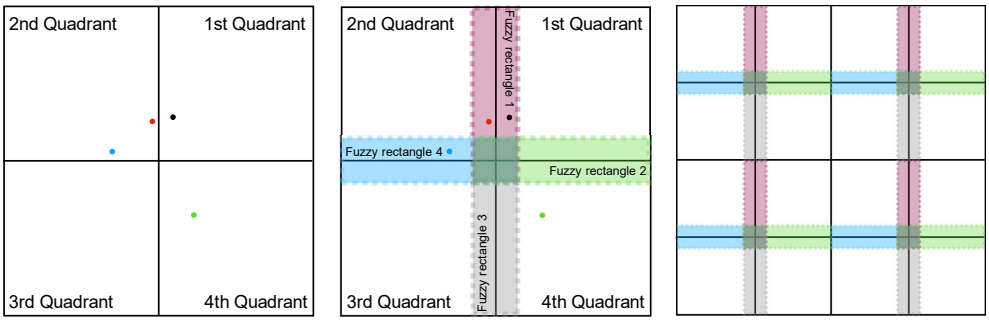
The RFQ method does not utilize the raw two-dimensional coordinates of nodes directly. It initiates by quadrisecting the entire 2D space, representing a node's quadrant membership via a 4-dimensional one-hot vector. The quadrant containing the

node is then recursively quadrisected, with each subsequent membership represented by another 4-dimensional one-hot vector. This recursive process is iterated k times. For instance, with $k = 6$, the outcome is a concatenated 24-dimensional binary vector composed of 6 one-hot vectors. A potential problem arises for two nodes in close proximity near a partition boundary may be placed in different quadrants, leading to disparate representations and introducing distortion in their relational encoding. To mitigate this boundary effect and more accurately reflect the relationships between such nodes, we incorporate a fuzzy representation concept. Centered on the four edges that delineate the quadrants, four rectangles (as depicted in Figure 2b for each edge) are defined. An additional 4-dimensional binary vector is generated to signify a node's membership within these fuzzy boundary regions. Thus, for $k = 6$, a supplementary 24-dimensional binary vector is produced.

In summary, at each stage of the recursive representation, two vectors are generated: a 4-dimensional one-hot vector denoting quadrant membership and a 4-dimensional binary vector for the fuzzy representation. This yields a composite vector of total dimensionality $8k$. Primary quadrant membership suggests a fundamental level of spatial proximity. The incorporation of fuzzy rectangles compensates for the potential loss of adjacency information inherent to hard partition boundaries. Through this recursive quadrisection process, each node progressively refines its corresponding quadrant. Consequently, the representation encodes multi-scale proximity: proximity within the entire space, within a $\frac{1}{4}$ space, within a $\frac{1}{16}$ space, ..., and finally within a $\frac{1}{4^{(k-1)}}$ space. This approach functions as a form of holographic representation. The adjacency information for nodes near boundaries is further enriched by the fuzzy rectangles. The width of these fuzzy rectangles was set to $\frac{1}{15}$ of the quadrant width, a value determined empirically through experiments detailed in Section 4.3.4. This representation paradigm provides a significantly richer input encoding compared to simplistic 2D coordinates or typical token embeddings derived solely from them.

Figure 2 illustrates the first two levels of RFQ. Figures 2a and 2b represent the first level, while Figure 2c shows the second level. This process continues for a total of k levels. Figure 2d visualizes the representation of the points from Figure 2b using quadrisection and fuzzy rectangles at $k = 1$. Increasing k in RFQ enhances the resolution of the representation, analogous to increasing the magnification of a microscope.

Node adjacency is captured through both quadrisection and fuzzification. For example, in the case of $k = 1$, nearby points may appear to be close within the same quadrant, but as k increases, a more detailed representation emerges. Figure 3 demonstrates an example where two relatively close nodes belong to the same quadrant when $k = 1$, but fall into different quadrants when $k = 4$. The representational efficacy of RFQ becomes particularly pronounced for large-scale instances (large N), as the average inter-node distance diminishes with increasing N. Consequently, the optimal maximum depth k is contingent upon the instance size. In the experiments detailed in this paper, the values $k = 5, 6, 8, 9$ were designated for $N = 50, 100, 500, 1000$, respectively—values determined empirically through the ablation studies in Section 4.3.4.



(a) Quadrisection (b) Recursive Fuzzy Quadrisection ($k = 1$) (c) Recursive Fuzzy Quadrisection ($k = 2$)

	1st Quadrant	2nd Quadrant	3rd Quadrant	4th Quadrant	Fuzzy rectangle 1	Fuzzy rectangle 2	Fuzzy rectangle 3	Fuzzy rectangle 4
●	0	1	0	0	1	0	0	0
●	0	1	0	0	0	0	0	1
●	0	0	0	1	0	0	0	0
●	1	0	0	0	1	0	0	0

(d) RFQ representation of the example points at $k = 1$, with each point encoded using one-hot vectors for the quadrants and fuzzy rectangles.

Fig. 2: Node representation encoding for example nodes via **Recursive Fuzzy Quadrisection (RFQ)**.

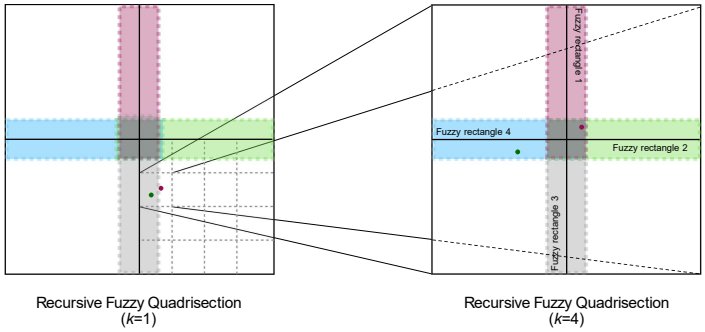
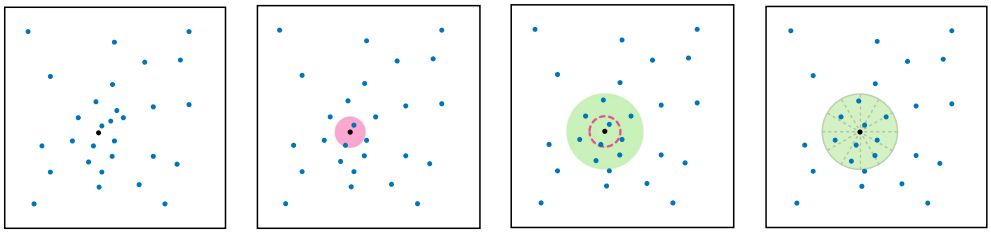


Fig. 3: Effect of k on **Recursive Fuzzy Quadrisection**. With $k = 1$ (left), two nearby nodes belong to the same quadrant, whereas with $k = 4$ (right), the finer partition separates them into different quadrants.

3.2.2 Densities around a Node

To incorporate richer spatial information, two types of feature vectors are generated based on the density around each node. An example of this process is illustrated in Figure 4. Figure 4a represents a given instance. Figures 4b and 4c show a discretized representation of the node counts within two concentric circles of different radii centered on a given node, encoded as a one-hot vector. This representation is referred to



(a) Instance (b) Directionless Density Embedding (c) Directionless Density Embedding (d) Directional Density Embedding

Fig. 4: Illustration of the proposed density embedding. (a) Original instance. (b–c) Directionless density embeddings with different neighborhood radii, capturing nearby and broader neighborhood information, respectively. (d) Directional density embedding, where neighboring points are grouped into 12 directional sectors to capture direction-specific dense and sparse regions.

as Directionless Density Embedding. Figure 4d depicts a node with 12 radial sectors, where the density within each sector is also expressed as a one-hot vector. This representation is referred to as Directional Density Embedding. For example, if each of these 14 types of densities (2 directionless + 12 directional) is discretized into four classes, they are concatenated to form a composite 56-dimensional binary vector.

For the directionless and Directional Density Embeddings, the radii must decrease as the instance size N increases, since a larger N leads to a higher density of nodes per unit area. The radii were designed such that, on average, the smaller circle (radius 1) encompasses approximately 3 nodes, while the larger circle (radius 2) contains roughly 16 to 22 nodes, as specified in Table 2. These radii were selected from empirical observations under the uniform distribution, where they consistently yield satisfactory performance. We note, however, that they are calibrated to the node density of the uniform distribution and may not perform well under other distributions; determining distribution-robust radii remains a limitation of the present work. Section 4.3.2 further demonstrates this limitation, showing that the same settings are not effective for nonuniform distributions.

Instance Size	Directionless Radius 1	Directionless Radius 2	Directional Radius
TSP-50	$\frac{7}{N} = 0.14$	$\frac{19}{N} = 0.38$	$\frac{19}{N} = 0.38$
TSP-100	$\frac{10}{N} = 0.1$	$\frac{25}{N} = 0.25$	$\frac{25}{N} = 0.25$
TSP-500	$\frac{22}{N} = 0.044$	$\frac{52}{N} = 0.104$	$\frac{52}{N} = 0.104$
TSP-1000	$\frac{31}{N} = 0.031$	$\frac{73}{N} = 0.073$	$\frac{73}{N} = 0.073$

Table 2: Density embedding details for TSP problem sizes.

3.3 Encoder

Following the embedding of node distribution by the Quad Embedder, spatial PE is injected into the embeddings prior to the first attention operation. This encoding, originally proposed in DETR (Carion et al., 2020) to imbue image patches with their 2D spatial context, extends the conventional Transformer (Vaswani, 2017)'s 1D sinusoidal PE to two dimensions, as formalized in Equation (1). This work represents its first application within the TSP domain, reflecting the two-dimensional relations of the vertices. The ablation studies in Section 4.3.4 confirmed a measurable performance improvement attributable to the incorporation of this spatial PE.

$$\text{Spatial PE}_{(i,\,k)} = \begin{cases} concat\Big(\sin\Big(\frac{x_i}{10000^{4\lfloor k/2 \rfloor/d}}\Big), \sin\Big(\frac{y_i}{10000^{4\lfloor k/2 \rfloor/d}}\Big)\Big), & \text{if } k \text{ is even,} \\ concat\Big(\cos\Big(\frac{x_i}{10000^{4\lfloor k/2 \rfloor/d}}\Big), \cos\Big(\frac{y_i}{10000^{4\lfloor k/2 \rfloor/d}}\Big)\Big), & \text{if } k \text{ is odd,} \end{cases} \tag{1}$$

We adopt the following notation throughout. Let N denote the number of nodes in an instance, L the number of encoder (equivalently, decoder) layers, and d the dimensionality of the instance embedding.

The encoder transforms the coordinates of an instance into a set of context-aware vertex embeddings. The Quad Embedder first projects the coordinates of node i onto a d-dimensional token embedding e_i (Equation (2)), to which a positional encoding $pe_{e,i}$ is added in order to form $x^1_{e,i}$, the input to the first encoder layer (Equation (3)). x^1_e denotes the stacked representation of all N nodes. Each of the L layers comprises two residual sublayers. The first applies multi-head self-attention (MHA) followed by layer normalization, yielding the intermediate representation h^l_e (Equation (4)); the second applies a position-wise FFN under a second residual connection, yielding the layer output z^l_e (Equation (5)). The output of layer l is taken as the input to layer $l+1$, so that z^L_e constitutes the final encoder representation (Equation (6)).

$$e_i = InputEmbedder(coordinates) \tag{2}$$

$$x^1_{e,i} = e_i + pe_{e,i}, \quad (i = 1, 2, \ldots, N) \tag{3}$$

$$h^l_e = \text{LayerNorm}(x^l_e + \text{MHA}(q = x^l_e, k = x^l_e, v = x^l_e)), \quad (l = 1, 2, \ldots, L) \tag{4}$$

$$z^l_e = \text{LayerNorm}(h^l_e + \text{FFN}(h^l_e)), \quad (l = 1, 2, \ldots, L) \tag{5}$$

$$x^{l+1}_e = z^l_e, \quad (l = 1, 2, \ldots, L-1) \tag{6}$$

The final output of the encoder, z^L_e, represents a comprehensive graph context of the encoded vertices of the graph. This representation is exploited in three distinct ways within our model. First, it is utilized as the input embedding for the decoder, where it is combined with cyclic PE and provided as input to the decoder's Masked MHA module. Second, it serves as the key and value in the decoder's cross-attention mechanism. Finally, it is employed as the foundational material for constructing a Context-Binding Matrix, enabling the decoder's output embedding to be matched with the most similar encoded vertex representation. Among these, the first and the third are, to our knowledge, novel.

3.4 Decoder

The decoder utilizes the final processed output embedding z_e^L from the encoder, enhanced with the CPE proposed in DACT (Ma et al., 2021), as its input. The CPE incorporates the cyclic property inherent in TSP solutions. This encoding is specifically engineered to reflect the rotational and reflectional invariance of TSP tours by ensuring the proximity between the start and end tokens of the sequence, thereby effectively representing the cyclic nature of the tour. The mathematical formulation of CPE is expressed in Equation (7).

$$\mathrm{CPE}_{(i,\,k)} = \begin{cases} \sin\left(\omega_k \cdot \left(\left(z(i) \bmod \frac{4\pi}{\omega_k}\right) - \frac{2\pi}{\omega_k}\right)\right), & \text{if } k \text{ is even,} \\ \cos\left(\omega_k \cdot \left(\left(z(i) \bmod \frac{4\pi}{\omega_k}\right) - \frac{2\pi}{\omega_k}\right)\right), & \text{if } k \text{ is odd,} \end{cases} \tag{7}$$

where $z(i) = \dfrac{i-1}{N} \cdot \dfrac{2\pi}{\omega_k} \cdot \left\lceil \dfrac{N+1}{2\pi/\omega_k} \right\rceil$ is evaluated separately for each dimension $k = 1, 2, \ldots, d$.

Using the encoder output, z_e^L, rather than random encoding or encoder's initial input $z_{e,j}^1$ as in typical Transformers, embodies the essential representation of the graph's context. Section 4.3.4 experimentally validates that leveraging z_e^L enhances performance over utilizing $z_{e,j}^1$.

The decoder constructs the tour autoregressively. Let V denote the number of nodes visited so far and $t_{pred}[j]$ the node selected at step j. At each decoding step j, the encoder output of the corresponding node, $z^L_{e,t_{pred}[j]}$, is retrieved and combined with the decoder positional encoding $pe_{d,j}$ (Equation (8)). x_d^l denotes the stacked representation of all V steps. Each of the L layers then comprises three residual sublayers. The first applies masked multi-head attention over the partial tour, which prevents each step from attending to subsequent ones, and yields h_{d1}^l (Equation (9)). The second applies cross-attention, in which the queries come from h_{d1}^l while the keys and values come from the encoder output z_e^L, enabling the partial tour to exploit the graph context (Equation (10)). The third applies a position-wise FFN under a further residual connection, yielding the layer output z_d^l (Equation (11)), which is taken as the input to the next layer (Equation (12)).

$$x_{d,j}^1 = z^L_{e,\,t_{pred}[j]} + pe_{d,j}, \quad (j = 1, 2, \ldots, V) \tag{8}$$

$$h_{d1}^l = \mathrm{LayerNorm}(x_d^l + \text{Masked-MHA}(q = x_d^l, k = x_d^l, v = x_d^l)), \quad (l = 1, 2, \ldots, L) \tag{9}$$

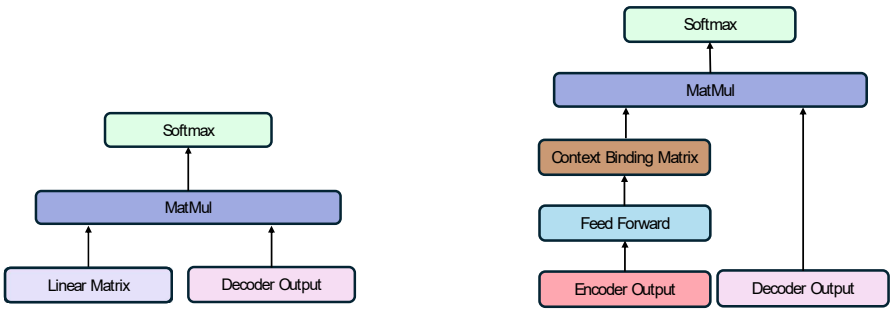
$$h_{d2}^l = \mathrm{LayerNorm}(h_{d1}^l + \mathrm{MHA}(q = h_{d1}^l, k = z_e^L, v = z_e^L)), \quad (l = 1, 2, \ldots, L) \tag{10}$$

$$z_d^l = \mathrm{LayerNorm}(h_{d2}^l + \mathrm{FFN}(h_{d2}^l)), \quad (l = 1, 2, \ldots, L) \tag{11}$$

$$x_d^{l+1} = z_d^l, \quad (l = 1, 2, \ldots, L-1) \tag{12}$$

3.5 Context-Binding Matrix : Vector-Flow Changes in Relation to Weak Token Identities

In the conventional Transformer (Vaswani, 2017) architecture, the linear matrix serves as a tool to measure similarity between the decoder output z_d^L and each candidate



(a) Original Transformer's Linear Matrix (b) Context-Binding Matrix

Fig. 5: Comparison between linear matrix of Transformer and Context-Binding Matrix of QuadFormer.

token by matrix multiplication. The resulting logits are then converted into a probability distribution over the next token via the softmax function (Figure 5a). This design is particularly well suited to language modeling, as in BERT (Kenton and Toutanova, 2019), ChatGPT (Ray, 2023), and LLaMA (Touvron et al., 2023), where each token has a globally consistent identity across the corpus. For instance, the word 'apple' retains its own meaning regardless of the sentence or document in which it appears. It is therefore natural to define a global word dictionary for the entire dataset and use it as the linear layer (Encoder LUT).

In contrast, in the TSP domain, where inputs are given as two-dimensional coordinates, a node cannot be assigned a globally consistent identity from its coordinates or index alone. The characteristics of a node are inherently instance-dependent: even nodes with the same index or coordinates may assume different roles across instances. TSP tokens therefore possess only a weak identity. Based on this observation, we hypothesize that the output projection should be dynamically constructed from the encoder outputs z_e^L.

The most direct realization of this hypothesis is to reuse the encoder output z_e^L as the output projection matrix. Since z_e^L provides an instance-conditioned representation for each input node, each decoding decision can be scored directly against the representations of the candidate nodes in the current instance. In our experiments, however, reusing the raw encoder output z_e^L directly proved suboptimal. A representational gap remains between z_e^L and the decoder output z_d^L, the latter having been abstracted through multiple decoder layers. We therefore transform z_e^L through a feed-forward network to better align it with the representation space of z_d^L. We refer to the resulting matrix as the **Context-Binding Matrix** W_c (Equation (13)). The output projection matrix is thus reconstructed for each instance, allowing every decoding decision to be conditioned on the context of the current instance.

$$W_c = \mathrm{FFN}(\mathrm{LayerNorm}(z_e^L)) \tag{13}$$

$$y_{pred} = \mathrm{softmax}\Big(z_d^L (W_c)^T + \mathrm{VisitedMask}\Big) \tag{14}$$

The Context-Binding Matrix is multiplied by the decoder output embedding z_d^L to measure the compatibility between the current decoding state and each candidate node (Figure 5b). The resulting scores serve as logits and are converted into a probability distribution over the next node via the softmax function (Equation (14)). The $VisitedMask$ prevents previously visited nodes from being selected again. All entries are initially set to zero, and the entry corresponding to each visited node is set to $-\infty$. Since the first node is given as the initial node of the decoding process, it is treated as already visited, and its corresponding mask entry is set to $-\infty$.

The Context-Binding Matrix is motivated by empirical observations, and we validate the preceding rationale through a controlled ablation study in which only the output projection matrix is varied (Table 8).

3.6 Open-Hypothesis Decoding at Inference

In the early stages of tour construction, limited state information induces high uncertainty regarding the long-term effects of node selections. In contrast, later decisions benefit from accumulated path information, which reduces this uncertainty. To mitigate the uncertainty present in the initial stage, we introduce **Open-Hypothesis Decoding** (OHD), a strategy that defers commitment at the critical first decision point where uncertainty is maximal. This method leverages the insight that partial optimality in the TSP does not imply global optimality, thereby allowing initially suboptimal choices to potentially lead to superior feasible tours.

Specifically, QuadFormer generates a probability distribution over all unvisited nodes from the starting node, selects the top m candidates based on their probabilities, and applies greedy decoding independently to each hypothesis (Figure 6). This expansion is performed exclusively at the first decision step, where the potential influence on the final tour quality is greatest. Thereafter, each of the m paths evolves independently using standard greedy selection, thereby achieving a balance between exploration during the high-uncertainty initial phase and computational efficiency during subsequent decisions. As shown in Table 3 and Table 4, introducing exploration ($m \geqq 2$) at this critical decision point substantially improves solution quality compared to the pure greedy approach ($m = 1$).

We conducted the following experiments to assess the effectiveness of OHD and identify a suitable value of m. The results are shown in Figures 7 and 8. The experiments were run on 128 instances each of TSP-500 and TSP-1000, rolling out tours from the top-10 candidates at 20 different starting points. For each dataset, we recorded the optimality gap of every candidate across 2,560 ($128 \times 20 \times 10$) rollouts.

For each rollout, we define the winning hypothesis as the one attaining the smallest optimality gap. Figure 7 reports the win rate at each first-step probability rank m, that is, how often the rank-m hypothesis turns out to be the winner. The horizontal axis denotes the first-step probability rank of a hypothesis, and the vertical axis denotes the fraction of cases in which the rank-m hypothesis becomes the winner. If the first-step probability were irrelevant to final quality, every bar would sit at the uniform level of 10%; if it were a perfect predictor, the bar at m=1 would reach 100%. As shown in Figure 7, the win rate at m=1 was only 18.2% on TSP-500 and 16.1% on TSP-1000. These values exceed the uniform expectation of 10%, so the first-step probability has

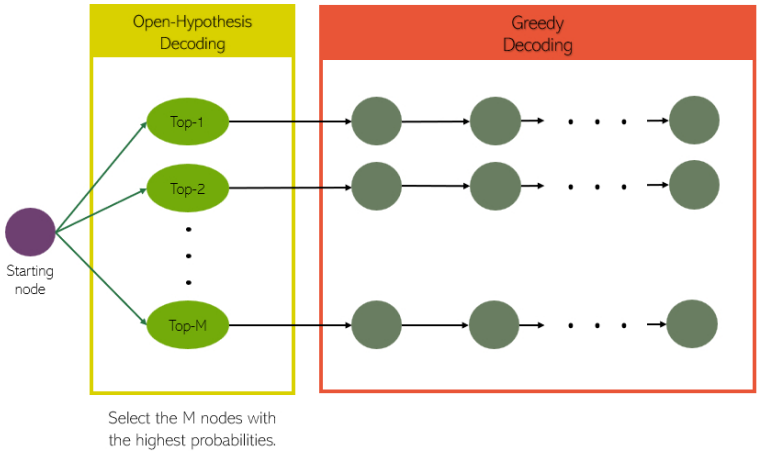
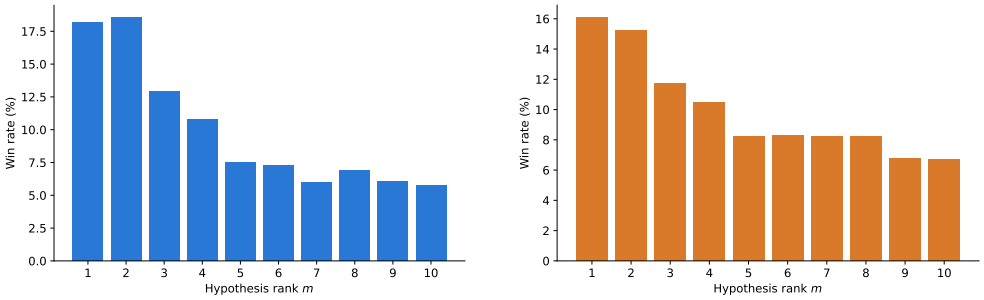


Fig. 6: In Open-Hypothesis Decoding, top M candidates are selected from the starting node, and greedy decoding is subsequently applied to complete each sequence.

some predictive value. Nonetheless, the winner was not the top-ranked hypothesis in roughly 82 to 84% of rollouts. The win rate moreover decreases only mildly as m increases, and even the lowest-ranked hypothesis (m=10) retains a win rate of 5.7 to 6.7%. The first-step probability is therefore a weak predictor of final solution quality, which supports diversifying the search through OHD rather than committing to the top-ranked hypothesis.

Figure 8 shows the gain $\Delta(m)$ obtained by expanding the hypothesis set from $m-1$ to m. The horizontal axis denotes the expansion step ($m-1 \rightarrow m$), and the vertical axis denotes the mean reduction in optimality gap due to the expansion. Figure 8 provides a quantitative basis for deciding how many hypotheses are worth retaining. The first expansion ($1 \rightarrow 2$) produced the largest gain, 1.72 to 2.01%, after which the gain fell to about 0.56% at $2 \rightarrow 3$, 0.28 to 0.33% at $3 \rightarrow 4$, and 0.18% at $4 \rightarrow 5$, shrinking below 0.13% from $5 \rightarrow 6$ onward. Notably, the marginal gain remained positive across all expansion steps. Expanding the hypothesis set therefore does not degrade performance. When computational resources permit, retaining more hypotheses consistently improves solution quality. However, because the computational cost grows linearly with the number of rollouts while the marginal gain diminishes, choosing an appropriate value of m is important under resource constraints.

Figure 9 measures inference time against optimality gap, showing that inference time grows as m increases. Balancing solution quality against computational cost, we therefore adopt $m = 4$. Furthermore, Figure 8 reveals that the marginal contribution of lower-ranked hypotheses persists longer as the instance size grows: from $2 \rightarrow 3$ onward, Δ on TSP-1000 stays at or above that on TSP-500. This suggests that the appropriate m may be an increasing function of problem size, and that raising m is worth considering for larger instances beyond TSP-1000.



(a) TSP-500 (b) TSP-1000

Fig. 7: Win rate of each hypothesis by first-step probability rank m. The top-ranked hypothesis wins only 16–18% of rollouts, indicating that the first-step probability is a weak predictor of final solution quality.

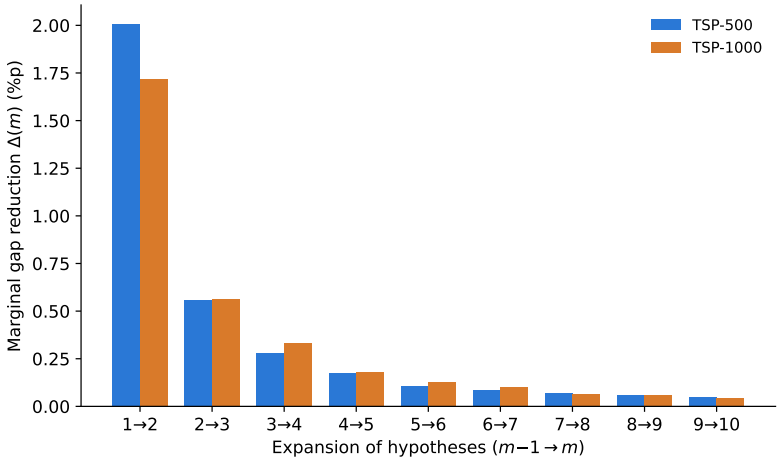
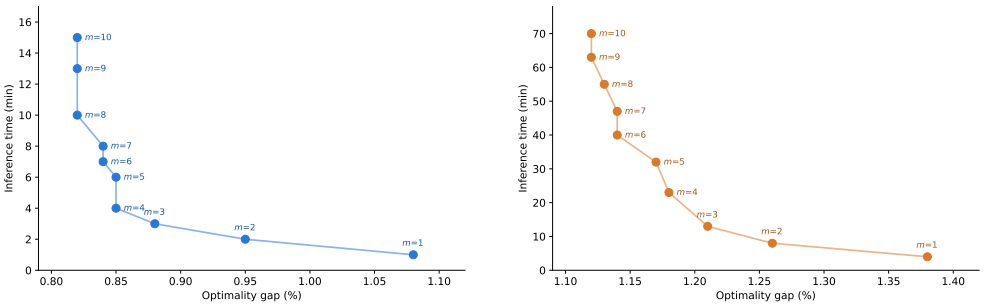


Fig. 8: Marginal reduction in the mean optimality gap when increasing the number of hypotheses from $m-1$ to m.

4 Experimental Results

4.1 Experiment Setting

The training data for the model was generated using the code from Fu et al. (2021). The dataset sizes align with those used in DIFUSCO (Sun and Yang, 2023) : 1,502,000 instances for TSP-50 and TSP-100, 128,000 instances for TSP-500, and 64,000 instances for TSP-1000. For testing, 1,280 instances were used for each of TSP-50 and TSP-100, and 128 instances for each of TSP-500 and TSP-1000. The inference time for TSP-50 and TSP-100 is reported as one-tenth of the actual measured seconds.



(a) TSP-500 (b) TSP-1000

Fig. 9: Optimality gap–inference time trade-off for different numbers of search candidates (hypotheses m)

The model architecture employs an embedding dimension of 256 and a feed-forward layer dimension of 1,024. Both the encoder and decoder consist of 6 layers each, with multi-head attention comprising 8 heads. The model was implemented using the PyTorch framework. During training, the batch size was set to 1,280 for TSP-50 and TSP-100, and 40 for TSP-500 and TSP-1000. For validation, batch sizes of 1,280 were used for TSP-50 and TSP-100, and 60 for TSP-500 and TSP-1000. The AdamW optimizer (Loshchilov and Hutter, 2017) was adopted with betas set to [0.9, 0.98]. Training was conducted for 140 and 150 epochs for TSP-50 and TSP-100, respectively, and for 500 and 470 epochs for TSP-500 and TSP-1000, respectively. The hardware configuration utilized the NVIDIA H100 GPU with 94GB of memory. For training, two GPUs were employed, while inference was conducted on a single GPU. The batch size was set to 1,280 for TSP-50 and TSP-100, and 128 for TSP-500 and TSP-1000. For comparative analysis, Concorde was executed on an AMD EPYC 9354 32-Core Processor system. Model performance was evaluated based on the optimality gap, calculated as follows:

$$\text{Optimality Gap } (\%) = \frac{\text{Prediction} - \text{Optimal}}{\text{Optimal}} \times 100 \tag{15}$$

We evaluated the performance of various state-of-the-art models, including POMO (Kwon et al., 2020), DACT (Ma et al., 2021), Pointerformer (Jin et al., 2023), LEHD (Luo et al., 2023), the model proposed in Luo et al. (2025), and INViT (Fang et al., 2024). Each model was obtained from their respective GitHub repositories and tested accordingly. For INViT, we utilized the model from their GitHub repository[1]. Since this study focuses on enhancing the Transformer architecture itself, inference was conducted with all post-processing procedures removed, including 2-Opt (DACT), Random Re-Construct (RRC; LEHD), and Parallel local ReConstruction (PRC; the model proposed in Luo et al. (2025)). We acknowledge that this setting may understate the performance of these models relative to the same models with post-processing integrated. For completeness, results with post-processing enabled are provided in the Appendix A. The evaluation sets were identical across all experiments, and all inference

[1]The checkpoint of INViT is available in this repository.

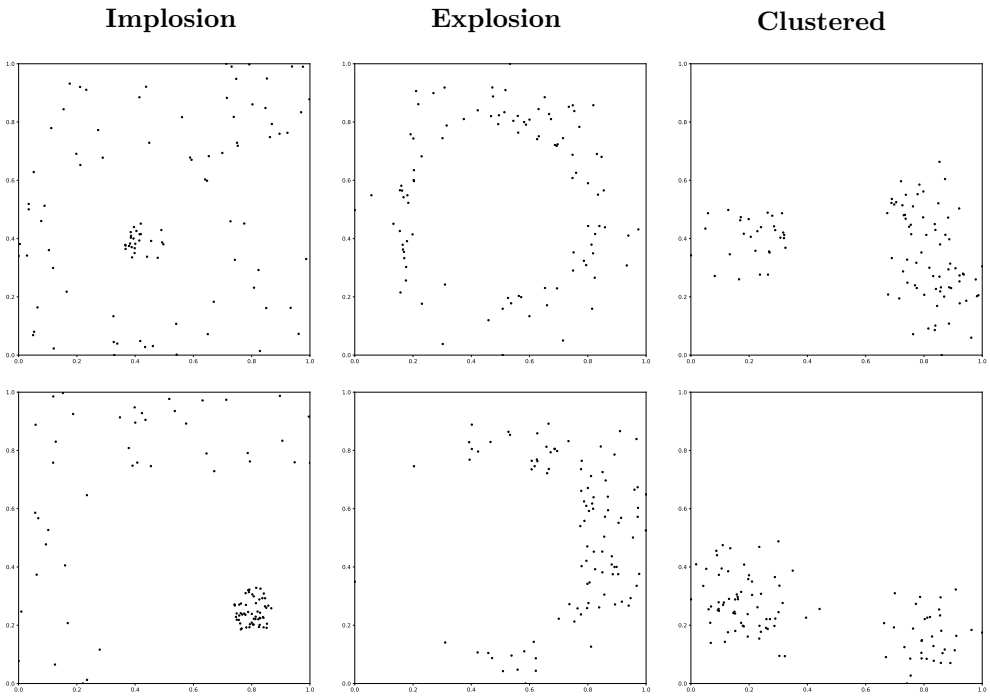


Fig. 10: Various distribution datasets: from left, the first column shows the implosion type, the second column the explosion type, and the third column the clustered type.

was conducted on the same hardware used for QuadFormer. As Concorde (Applegate et al., 2007) reports results only up to six decimal places, the optimality gap may be slightly larger compared to other models that employ floating-point representations. Consequently, apparently weird negative optimality gaps may occasionally occur. We report the average tour length and optimality gap across all instances.

4.2 Datasets

This study employed two distinct datasets for experimental evaluation. First, we constructed a random TSP dataset with random distribution within a unit square $[0, 1]$, utilizing the code from Fu et al. (2021). For label generation, we obtained optimal solutions through the Concorde (Applegate et al., 2007) algorithm for both training and test data on TSP-50, TSP-100, and TSP-500. For TSP-1000, due to time constraints, we employed the LKH3 algorithm for training data while using Concorde for test data. Second, we utilized datasets with various distributions, specifically the implosion, explosion, and clustered data with N=100 from the Fang et al. (2024), using 100 instances of each for evaluation, as illustrated in Figure 10. Third, we incorporated the TSPLIB dataset, which was originally introduced by Reinelt (1991) and comprises instances with diverse node distributions.

Table 3: Performance comparison on TSP-50 and TSP-100. We report the tour length, optimality gap and total inference time. Hypotheses (m) denotes the branching factor in our Open-Hypothesis Decoding (OHD). Note that we highlight the **overall best** and second-best results across all methods..

Algorithm	m	TSP-50			TSP-100		
		Length	Gap (std)	Time	Length	Gap (std)	Time
Concorde	-	5.69	0.00	6s	7.76	0.00	31.2s
Single-Start Inference							
POMO	-	5.71	0.34 (0.75)	<1s	7.83	0.94 (0.81)	<1s
POMO (8aug)	-	5.69	0.05 (0.19)	<1s	7.78	0.31 (0.38)	<1s
DACT (T=0)	-	23.27	7.01 (0.57)	<1s	24.74	9.68 (0.57)	<1s
Pointerformer	-	5.71	0.34 (0.84)	<1s	7.86	1.20 (1.57)	<1s
LEHD	-	5.72	0.54 (1.02)	<1s	7.80	0.51 (0.68)	1.8s
Luo et al. (2025)	-	6.22	9.36 (6.86)	<1s	8.12	4.63 (3.92)	<1s
QuadFormer	-	5.70	0.18 (0.57)	<1s	7.81	0.66 (1.42)	<1s
	2	5.69	0.07 (0.19)	<1s	7.78	0.26 (0.44)	<1s
	3	5.69	0.05 (0.15)	<1s	7.78	0.21 (0.39)	<1s
	4	5.69	0.05 (0.15)	<1s	7.77	0.20 (0.32)	<1s
Multi-Start Inference							
POMO	-	5.69	0.09 (0.26)	<1s	7.79	0.39 (0.46)	<1s
POMO (8aug)	-	5.69	0.00 (0.12)	<1s	7.77	0.11 (0.25)	<1s
Pointerformer	-	5.69	0.10 (0.28)	<1s	7.79	0.46 (0.49)	<1s
INViT-3V	-	5.70	0.25 (0.26)	1m 11s	7.85	1.17 (0.56)	3m 20s
QuadFormer	-	5.69	0.01 (0.07)	<1s	7.76	0.04 (0.12)	1.8s
	2	5.69	0.00 (0.07)	1s	7.76	0.01 (0.09)	3.7s
	3	5.69	**-0.01** (0.05)	1.5s	7.76	0.01 (0.09)	5.5s
	4	5.69	**-0.01** (0.05)	2s	7.76	**0.00** (0.08)	7.3s

4.3 Experimental Results

4.3.1 Experiments on Random TSP

Performance was evaluated based on the optimality gap, with optimal solutions emphasized in bold and runner-up results underlined. Two primary decoding strategies were examined: single-start inference, where decoding initiates from a single starting node, and multi-start inference, as proposed in POMO (Kwon et al., 2020). In the multi-start inference framework, solutions are generated from multiple different starting nodes, and the best solution among them is selected as the output. Notably, LEHD (Luo et al., 2023) and the model proposed by Luo et al. (2025) do not implement multi-start inference, as they instead employ post-processing techniques such as RRC and PRC to enhance solution quality. For multi-start inference, POMO and Pointerformer (Jin et al., 2023) each conduct N independent inference runs per instance, where N corresponds to the number of nodes; each run initiates from a unique starting node, ensuring that all possible starting points are explored. In contrast, INViT

(Fang et al., 2024) follows its original protocol by using N distinct starting points for TSP-50 and TSP-100, and 100 distinct starting points for TSP-500 and TSP-1000. QuadFormer utilizes a fixed strategy, performing inference from 15 different starting nodes across all instance sizes.

As shown in Table 3, QuadFormer coupled with OHD exhibits near-perfect performance on small instances. Specifically, it achieves the optimal solution for TSP-50 when $m \geq 2$ and for TSP-100 when $m = 4$.

On large-scale instances such as TSP-500 and TSP-1000, baseline models including LEHD and the approach of Luo et al. (2025) demonstrate strong performance even under single-start inference. This robustness stems from their training paradigm, which samples partial tours of varying lengths. Such sampling exposes the model to a diverse set of starting conditions, enabling stronger generalization and yielding single-start performance levels comparable to what is typically achieved through multi-start inference. This mechanism appears to mitigate a well-recognized limitation of Transformer architectures—their tendency for subsequent decoding decisions to depend heavily on earlier ones—by encouraging a more diverse exploration of the solution space. This observation presents an important research direction, and we plan to investigate integrating similar strategies into future extensions of our model.

In contrast, QuadFormer exhibits a relatively higher optimality gap under simple greedy decoding ($m = 1$) compared to these models. Notably, this gap diminishes rapidly as the number of hypotheses m in OHD increases. Specifically, increasing m from 1 to 4 reduces the gap for TSP-500 from 4.44% to 1.98%, and for TSP-1000 from 4.83% to 2.44%, representing approximately a twofold improvement. These results demonstrate that our strategy of deferring commitment during the initial, most uncertain stage effectively controls uncertainty during inference, thereby substantially improving solution quality.

As shown in Table 4, QuadFormer achieves the most competitive optimality gap among all compared methods for large-scale instances when employing the OHD decoding strategy under multi-start inference. With respect to inference time, QuadFormer requires greater computational resources than lightweight baselines such as Pointerformer, LEHD and the model proposed in Luo et al. (2025). This increased cost is due to our use of the full six-layer encoder/decoder architecture as described by Vaswani (2017), rather than a shallower encoder/decoder. This design maximizes model expressiveness and allows for the full leverage of rich spatial features derived from RFQ and density embeddings. Although this architectural choice leads to longer inference times, the results in Tables 3 and 4 demonstrate that this additional investment consistently yields superior solution quality.

Meanwhile, the fastest inference time on TSP-1000 is observed for the model Luo et al. (2025), which significantly reduces computational complexity by employing lightweight cross-attention ($O(N)$) rather than full self-attention ($O(N^2)$). Integrating such efficient attention mechanisms for scalable problem solving remains an important direction for future development of our model.

The statistical significance of the results reported in Tables 3 and 4 is discussed in Appendix B.

Table 4: Performance comparison on TSP-500 and TSP-1000. We report the tour length, optimality gap and total inference time. m denotes the branching factor in our Open-Hypothesis Decoding (OHD). Note that we highlight the **overall best** and second-best results across all methods.

Algorithm	m	TSP-500			TSP-1000		
		Length	Gap (std)	Time	Length	Gap (std)	Time
Concorde	-	16.58	0.00	40m	23.23	0.00	9h
Single-Start Inference							
Pointerformer	-	17.64	6.46 (1.24)	3s	25.55	10.04 (1.14)	17s
LEHD	-	16.80	1.30 (0.66)	6s	23.84	2.64 (1.12)	37s
Luo et al. (2025)	-	16.87	1.73 (1.24)	4s	23.57	1.48 (1.19)	13s
QuadFormer	-	17.32	4.44 (3.91)	4s	24.35	4.83 (3.30)	17s
	2	17.05	2.83 (2.27)	8s	24.02	3.42 (2.51)	35s
	3	16.97	2.34 (1.82)	12s	23.91	2.96 (2.03)	52s
	4	16.91	1.98 (1.28)	16s	23.79	2.44 (2.27)	1m 9s
Multi-Start Inference							
Pointerformer	-	17.19	3.64 (0.48)	3s	24.92	7.24 (0.51)	17s
INViT-3V	-	17.36	4.89 (0.52)	12m	24.49	5.94 (0.40)	29m
QuadFormer	-	16.76	1.08 (0.39)	1m	23.55	1.38 (0.34)	4m
	2	16.74	0.95 (0.34)	2m	23.52	1.26 (0.29)	8m
	3	16.73	0.88 (0.30)	3m	23.51	1.27 (0.27)	13m
	4	16.73	**0.85** (0.29)	4m	23.50	**1.18** (0.27)	17m

Table 5: Experimental results on various distributions.

	Implosion		Explosion		Clustered	
	Length	Gap (std)	Length	Gap (std)	Length	Gap (std)
LEHD	7.20	1.19 (1.71)	6.61	0.63 (0.78)	5.41	**1.56 (2.16)**
Luo et al. (2025)	7.59	6.81 (5.69)	6.94	5.81 (7.25)	5.62	5.48 (5.26)
INViT-3V	7.21	1.32 (0.66)	6.65	1.17 (0.53)	5.42	1.62 (0.68)
QuadFormer	7.13	**0.30 (1.42)**	6.60	**0.53 (1.96)**	5.68	7.35 (11.46)

4.3.2 Experiments on Various Distribution Dataset

In Table 5, we evaluate various models on datasets with diverse distributions used in INViT (Fang et al., 2024). For each distribution, 100 datasets with instance size 100 were prepared. Both INViT and QuadFormer performed multi-start inference using 100 distinct starting points per instance. As noted previously, LEHD (Luo et al., 2023) does not implement multi-start inference and was therefore evaluated using single-start inference alone. QuadFormer reports results using Open-Hypothesis Decoding with the number of hypotheses fixed at $m = 4$ during inference.

INViT was specifically designed to preserve generalization across diverse data distributions, and consequently exhibits the smallest performance gap across test sets drawn from different distributions. LEHD also achieved competitive performance across diverse datasets, and outperformed all other baselines on clustered instances. This can be attributed to LEHD's learning to reconstruct partial solutions, which enhances the model's generalization ability. In contrast, the model proposed in Luo et al. (2025) underperformed relative to the other baselines, as it is designed for solving large-scale instances, which may limit its effectiveness on the datasets considered here. QuadFormer achieved the best performance on implosion and explosion datasets but exhibited the lowest performance on clustered data. This performance difference stems from QuadFormer's design assumption of uniform data distribution. Clustered data features nodes densely concentrated in specific regions, while other areas are highly sparse, a nonuniform spatial distribution that QuadFormer struggles to capture effectively. Specifically, the radius parameter of the density embedding is fixed relative to the number of nodes N, and its size critically influences performance. Empirically, modifying the radii to $radius1 = 0.04$ and $radius2 = 0.23$ during inference reduced the optimality gap by 6.97%, indicating a limitation of fixed-radius approaches and highlighting the necessity of adapting radius parameters to data distribution characteristics.

In contrast, INViT overcomes this limitation by utilizing the k-nearest neighbors (k-NN) algorithm instead of fixed radii. INViT selects the k closest nodes to the current node, independent of absolute distances, concatenates their embeddings, and uses these as key and value inputs. This method treats relatively close nodes as neighbors regardless of absolute distance, enabling natural capture of dense regions within narrow ranges and sparse regions over broader ranges. This adaptive neighbor selection mechanism likely accounts for INViT's superior performance over QuadFormer on clustered datasets.

4.3.3 Experiments on TSPLIB

QuadFormer, trained on TSP-100, TSP-500, and TSP-1000 datasets, was used in an ensemble for testing, and the results are presented in Table 6. During multi-start inference, QuadFormer fixes the number of hypotheses in OHD at $m = 4$. The best performance is indicated in bold. Figure 12 illustrates instances ($rat783$, $pcb1173$, $u1432$) where QuadFormer exhibited poor performance, while Figure 11 depicts instances ($lin318$, $rd400$, $pcb442$) where QuadFormer achieved relatively good performance. A comparison of Figure 12 and Figure 11 reveals that high-performing instances display nodes evenly distributed across the entire coordinate plane, whereas low-performing instances exhibit nodes clustered within specific regions.

QuadFormer was trained specifically on TSP-100, TSP-500, and TSP-1000 instances generated from uniform distributions, demonstrating stable performance on instances with characteristics similar to the training distribution. In contrast, instances such as $rat783$, $pcb1173$, and $u1432$—where nodes are concentrated in portions of the coordinate plane—exhibit increased spatial concentration per unit area. This scenario resembles handling larger instances under uniform distributions (e.g., TSP-1500 or TSP-2000), in which the model fails to achieve satisfactory performance.

Table 6: Results on TSPLIB instances

	QuadFormer		INViT-3V		LEHD		Luo et al. (2025)	
Instance	Length	Gap	Length	Gap	Length	Gap	Length	Gap
eli51	6.22	**-0.26**	6.30	1.17	6.33	1.64	6.77	8.68
berlin52	4.34	**0.00**	4.34	0.04	4.34	0.03	4.82	11.09
st70	6.77	**-0.22**	6.85	0.89	6.81	0.33	6.91	1.87
pr76	5.52	0.96	5.53	1.28	5.47	**0.22**	5.58	2.11
eil76	7.16	**-0.19**	7.23	0.74	7.36	2.54	7.40	3.07
rd100	8.06	**0.00**	8.16	1.30	8.06	0.01	8.40	4.22
eil101	8.32	**-0.23**	8.57	2.70	8.53	2.31	8.69	4.12
bier127	6.43	9.78	6.05	3.24	6.14	4.76	6.03	**2.92**
ch130	8.75	**0.07**	9.16	4.76	8.79	0.56	9.64	10.26
pr136	7.52	12.09	6.8	1.43	6.74	**0.45**	7.30	8.89
ch150	9.34	**-0.02**	9.56	2.40	9.39	0.55	9.53	2.08
rat195	8.21	3.74	8.18	3.36	8.02	**1.42**	8.08	2.18
ts225	8.56	8.12	8.16	3.12	7.94	**0.28**	8.10	2.29
a280	9.32	3.72	9.59	6.79	9.25	3.02	9.11	**1.40**
lin318	10.55	1.79	10.73	3.49	10.51	**1.41**	11.05	6.58
rd400	15.45	**0.89**	16.14	5.37	15.47	1.00	15.66	2.25
pcb442	13.51	1.06	13.96	4.44	13.78	3.11	13.47	**0.77**
rat575	14.32	5.14	14.44	5.96	14.04	3.02	13.80	**1.32**
rat783	19.17	25.69	16.13	5.74	15.85	3.91	15.56	**1.99**
pcb1173	20.61	24.95	17.69	**7.24**	17.81	8.01	17.78	7.82
u1432	19.73	10.90	18.65	4.81	18.38	3.34	18.80	5.70

Instances from the same series (e.g., *rat*575 and *rat*783) share similar distributional characteristics but differ in spatial concentration, enabling performance comparisons within series to isolate its impact. For the *rat* series, *rat*575—closest to TSP-500 by size alone—achieves its best performance with the TSP-1000 model (among those trained on TSP-100, TSP-500, and TSP-1000), as its spatial concentration aligns more closely with TSP-1000 under uniform distributions. Conversely, *rat*783's spatial concentration exceeds that of TSP-1000, yielding degraded performance. Similar trends hold for the *pcb* series: *pcb*442, with evenly distributed nodes, demonstrates strong performance under the TSP-500 model. However, *pcb*1173—with nodes heavily clustered in portions of the coordinate plane and thus elevated spatial concentration—exhibits degraded performance even when inferred with the TSP-1000 model. These findings suggest that the model may face challenges adapting to elevated spatial concentrations unseen during training.

When the minimal bounding box enclosing all nodes approximates a square, rescaling input coordinates prior to inference can partially mitigate this limitation. Scaling coordinates to the range $[0, 1]$ while preserving aspect ratio redistributes clustered nodes more evenly across the plane, creating conditions akin to the training distribution (Figure 13). Indeed, for *u*1432, rescaling by a factor of 1.62 along the x- and y-axes improved the optimality gap from 10.09% to 6.91%. However, for instances like *rat*783 or *pcb*1173, where the bounding box approximates an elongated rectangle, achieving even node distribution remains challenging, rendering the approach's benefits negligible.

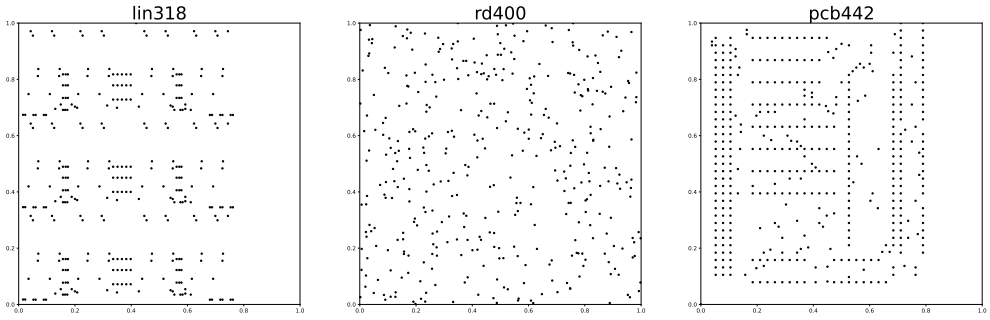


Fig. 11: Examples of TSPLIB instances where QuadFormer achieved relatively good performance include lin318, rd400, and pcb442 from left to right.

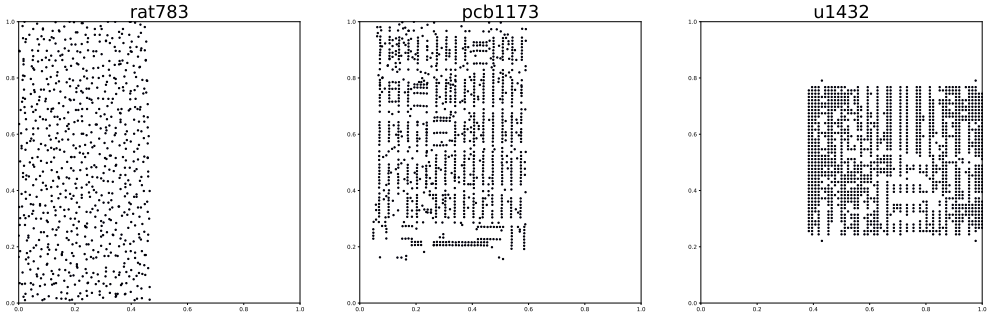


Fig. 12: Examples of TSPLIB instances where QuadFormer exhibited relatively weak performance include rat783, pcb1173, and u1432.

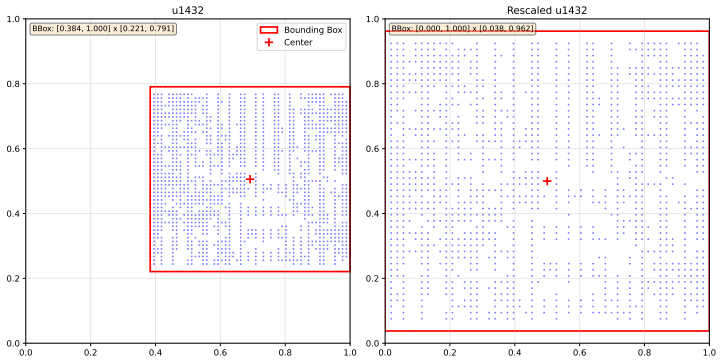


Fig. 13: The result of rescaling $u1432$. The original bounding box $[0.384, 1.000] \times [0.221, 0.791]$ (left) is rescaled by a factor of 1.62 along both axes, producing a new bounding box of approximately $[0.000, 1.000] \times [0.038, 0.962]$ (right) that fully covers the x-axis. As a result, the clustered nodes are distributed more evenly across the plane.

Table 7: Effect of excluding individual embedding components.

Model configuration	Gap
w/o recursive fuzzy quadrisection	1.77
w/o fuzzy rectangle	1.75
w/o directionless density	1.89
w/o directional density	1.75
w/o 2d coordinates	1.78
w/o augmentation and arctan($\frac{y}{x}$)	1.97
QuadFormer	**1.67**

4.3.4 Experiments on Architecture Design Components

This section presents experiments to demonstrate the efficacy of each component in QuadFormer and to determine hyperparameter values. To compare various conditions, experiments were conducted with reduced training data and model size. TSP-100 was trained for 100 epochs using 100,000 instances, while TSP-500 and TSP-1000 were trained for 200 epochs using 20,000 instances. The embedding dimension was set to 128, and the feed-forward layer dimension was configured to 512. In single-start inference, QuadFormer sets the hypothesis count in OHD to $m = 4$.

Experiments on Architecture Design Components

Table 7 examines the impact of removing spatial representation elements from QuadFormer. RFQ, feature of directionless density, and feature of directional density all demonstrated considerable influence. Table 8 evaluates the effectiveness of the Context-Binding Matrix. When the encoder LUT (linear layer) is used as the output projection matrix, as in the original Transformer, the model achieves an optimality gap of 2.54. Replacing the encoder LUT with the encoder output reduces the gap to 1.99. Applying an additional feed-forward transformation to construct the Context-Binding Matrix further improves the gap to 1.67. Table 9 investigates various configurations of PE in the encoder and decoder. Comparing the performance of configurations a and c in Table 9 demonstrates the effectiveness of spatial PE. Comparing configurations b and c reveals the effectiveness of CPE.

Hyperparameter Tuning Experiments

Fuzzy rectangles provide information about vertices that are located close to each other but may be classified into different regions due to their position at quadrant boundaries. Performance varies according to this width, and Table 10 presents experimental results from adjusting the fuzzy rectangle width in RFQ. Empirically, $\frac{1}{15}$ yielded the best performance. The quadrisection depth should vary according to instance size. As instance size increases, finer granularity is required for encoding, necessitating a corresponding increase in the quadrisection depth. Table 11 shows experimental results from varying the RFQ depth parameter k according to TSP instance size. The optimal depths for TSP 50, 100, 500, and 1000 were 5, 6, 8, and 9, respectively.

Table 8: Empirical validation of the **Context-Binding Matrix**. Replacing the encoder LUT with the encoder output reduces the optimality gap, while an additional feed-forward transformation yields further improvement.

Configuration	Gap
Encoder LUT (w/o Context-Binding Matrix)	2.54
Encoder output (w/o Context-Binding Matrix)	1.99
QuadFormer	**1.67**

Table 9: Experimental results under different configurations of positional encoding in the encoder and decoder.

Models	Enc PE	Dec PE	Gap
a	sinusoidal	cyclic	1.85
b	spatial	sinusoidal	1.88
c (QuadFormer)	spatial	cyclic	**1.67**

Table 10: Experimental results according to the variation in the width of the fuzzy rectangle.

Width of a fuzzy rectangle	Gap
1/17	1.69
1/15 (QuadFormer)	**1.67**
1/13	1.80

5 Conclusion

In this study, we proposed QuadFormer to enhance the geometric representational power of Transformer-based TSP solvers. By integrating Recursive Fuzzy Quadrisection (RFQ) and Density Embeddings with the architecturally novel Context-Binding Matrix, we establish a framework that effectively bridges the gap between raw data and the latent space (i.e., the space of learned representations). Our approach achieves near-optimal performance on small instances and demonstrates competitive results on large-scale problems.

Our experiments reveal three fundamental insights for neural combinatorial optimization. First, enhanced geometric embeddings—the combination of RFQ and density embeddings—provide rich spatial information that extends well beyond simple coordinate information. RFQ enables models to capture node topology across various spatial granularities through hierarchical partitioning, while density embeddings characterize local node distribution patterns, together allowing the model to perceive spatial structure from multiple perspectives. Second, the Context-Binding Matrix addresses the weak token identity problem inherent to the TSP domain. By explicitly projecting the encoder's graph representation into the decoder, it establishes a strong encoder-decoder coupling tailored to TSP's structural characteristics, which fundamentally differ from language models. Third, Open-Hypothesis Decoding (OHD) demonstrates the value of managing uncertainty during inference. Exploring only 3-4 alternatives at the initial decision reduces optimality gaps in single-start inference

Table 11: An experiment conducted by adjusting the depth of the RFQ to find the optimal granularity corresponding to different instance sizes.

Instance size	Depth	Gap
TSP-50	4	0.58
	5	**0.56**
	6	0.59
TSP-100	5	1.77
	6	**1.67**
	7	1.78
TSP-500	7	10.84
	8	**9.04**
	9	9.95
TSP-1000	8	13.66
	9	**13.28**
	10	13.71

from 4.44% to 1.98% on TSP-500 and from 4.83% to 2.44% on TSP-1000, indicating that selective exploration at critical junctures can yield substantial improvements while preserving computational efficiency.

QuadFormer attains the best results under multi-start inference, reaching optimality gaps of 0.90% on TSP-500 and 1.22% on TSP-1000, while baseline models such as LEHD (Luo et al., 2023) and Luo et al. (2025) achieve competitive performance under single-start inference through partial tour sampling. These findings validate the effectiveness of our design philosophy, yet they also highlight several clear challenges that warrant further investigation. Retaining the standard Transformer architecture and OHD to capture fine-grained geometric dependencies inevitably increases inference time compared to lightweight models. Incorporating the lightweight strategies demonstrated in Pointerformer (Jin et al., 2023) and Boost, or applying the partial tour sampling techniques employed by LEHD and Luo et al. (2025) during training, may simultaneously improve computational efficiency and generalization across diverse data distributions. In addition, integrating post-processing methods such as 2-Opt (Croes, 1958) or Monte Carlo Tree Search (Coulom, 2006) could further refine the quality of the obtained solutions. The values adopted in this study were empirically determined under the assumption of a uniform distribution and may therefore be further optimized. To enhance robustness across instances from diverse distributions, the fixed-radius neighborhood in our proposed model could be replaced with the k-NN approach used in INViT.

In conclusion, QuadFormer demonstrates that enriched embeddings combined with a suitably adapted architecture can meaningfully advance Transformer-based TSP

solvers. Our design choices—a deeper architecture to support expressive representations and sophisticated geometric encodings—indeed incur higher computational costs, but this trade-off reflects a deliberate commitment to addressing complex spatial optimization problems. Extending these spatial representation techniques and architectural insights to other NP-Hard optimization tasks that rely on geographical information represents a natural and promising direction for future research.

Declarations

- Funding : This research was supported by the Challengeable Future Defense Technology Research and Development Program through the Agency for Defense Development (ADD), funded by the Defense Acquisition Program Administration (DAPA) in 2025 (No. 915110201). This work was also supported by the BK21 FOUR Intelligence Computing Program (Department of Computer Science and Engineering, Seoul National University), funded by the National Research Foundation of Korea (NRF) (No. 4199990214639). The ICT at Seoul National University provides research facilities for this study.

- Conflict of interest/Competing interests (check journal-specific guidelines for which heading to use) : Not applicable
- Ethics approval and consent to participate : Not applicable
- Consent for publication : Not applicable
- Data availability : All source code, datasets, and trained models are publicly available at https://github.com/yookje/QuadFormer.git.
- Materials availability : All source code, datasets, and trained models are publicly available at https://github.com/yookje/QuadFormer.git.
- Code availability : All source code, datasets, and trained models are publicly available at https://github.com/yookje/QuadFormer.git.
- Author contribution : Jieun Yook conceived the model, designed the overall methodology, and took the lead in both the experimental implementation and the writing of the manuscript. Junpyo Seo collaborated on the experimental implementation with Jieun Yook.

If any of the sections are not relevant to your manuscript, please include the heading and write 'Not applicable' for that section.

Appendix A Performance Comparison with Post-Processed Baselines

In this section, we compare QuadFormer with each baseline under the postprocessing procedure originally associated with that method. For TSP-50 and TSP-100, we report results for DACT with 2-Opt, LEHD with Random ReConstruction (RRC), and the model of Luo et al. (2025) with Parallel ReConstruction (PRC) in Table A1. For TSP-500 and TSP-1000, we report results for LEHD with RRC and the model proposed in the model from Luo et al. (2025) with PRC in Table A2. In contrast, QuadFormer uses single-start decoding without any additional postprocessing. All experiments were

conducted on a single NVIDIA H100 94GB GPU to ensure a consistent hardware setting across methods.

The model proposed in Luo et al. (2025) is designed to solve large scale TSP instances, ranging from TSP-1000 to TSP-100,000, by progressively improving an efficiently constructed initial solution. It first generates an initial tour using random insertion and then refines the tour through repeated PRC steps. Specifically, PRC partitions the complete tour into partial tours of length r and reconstructs the internal nodes of each partial tour while fixing its two endpoints. Because the endpoints remain unchanged, each reconstruction preserves tour feasibility. Moreover, non-overlapping partial tours can be reconstructed in parallel, which substantially reduces the computational cost of the refinement process. Thus, rather than constructing the final solution in a single decoding step, the method progressively improves an initial solution through repeated local reconstruction.

Following the original design of Luo et al. (2025), we initialized each solution using random insertion and applied PRC for three iterations. For DACT, we applied 2-Opt for 1,000 iterations, whereas for LEHD, we applied one iteration of RRC.

In Tables 3 and 4, we set r to the full tour length for the model proposed in Luo et al. (2025). Under this configuration, the model fixes only the start and end nodes and reconstructs all remaining nodes in a single pass. Consequently, the solution is generated without random insertion or repeated partial reconstruction. This setting therefore isolates the model's direct solution construction capability from the effect of iterative postprocessing.

Postprocessing methods such as 2-Opt, RRC, and PRC generally improve solution quality as the number of refinement iterations increases, but they also incur additional inference cost. The more iterations are applied, the greater the potential improvement in solution quality, but the longer the inference time becomes. Accordingly, the baseline methods may achieve further improvements if additional refinement iterations are allowed.

As shown in Table A1 and Table A2, QuadFormer remains competitive with baselines that employ iterative postprocessing, despite using only single-start decoding and no additional refinement. These results indicate that QuadFormer achieves comparable solution quality without relying on repeated local improvement procedures.

Table A1: Comparison of QuadFormer against post-processing baselines on TSP-50 and TSP-100. QuadFormer results are reported using single-start decoding.

	TSP-50			TSP-100		
Method	Length	Gap (std)	Time	Length	Gap (std)	Time
DACT ($T = 1000$)	5.70	0.13 (0.45)	<1s	7.89	1.62 (1.11)	<1s
LEHD (RRC)	5.70	0.23 (0.55)	<1s	7.79	9.33 (0.53)	<1s
Luo et al. (2025) (PRC)	5.94	4.09 (2.88)	<1s	8.03	3.53 (2.06)	<1s
QuadFormer ($m = 4$)	5.69	**0.05** (0.15)	<1s	7.77	**0.20** (0.32)	<1s

Table A2: Comparison of QuadFormer against post-processing baselines on TSP-500 and TSP-1000. QuadFormer results are reported using single-start decoding.

	TSP-500			TSP-1000		
Method	Length	Gap (std)	Time	Length	Gap (std)	Time
LEHD (RRC)	16.79	**1.23** (0.63)	7s	23.80	2.49 (1.08)	37s
Luo et al. (2025) (PRC)	17.19	3.68 (0.84)	22s	24.11	3.82 (0.68)	1m 20s
QuadFormer ($m = 4$)	16.91	1.98 (1.28)	16s	23.79	**2.44** (2.27)	1m 9s

Appendix B Statistical Stability Analysis

We evaluate QuadFormer against each baseline in terms of average performance, worst-case robustness, and paired statistical significance across test instances. QuadFormer uses multi-start inference, performing inference from 20 different starting vertices and reporting the best optimality gap among the resulting solutions. All reported metrics are based on the optimality gap, defined as the percentage by which the obtained tour length exceeds the optimal tour length; thus, lower values indicate better performance. *Avg Gap* and *Max Gap* denote the mean and maximum optimality gaps, respectively, over the test instances. *Win/Tie/Loss* reports the number of instances for which QuadFormer achieves a lower, equal, or higher optimality gap than the baseline, respectively. *Wilcoxon p* reports the two-sided Holm-corrected p-value from a Wilcoxon signed-rank test between QuadFormer and the baseline, with values below 0.05 indicating a statistically significant difference. Negative optimality gaps arise from the six-decimal rounding of Concorde's optimal tour lengths, whereas the predicted tour lengths are reported at full precision.

As shown in Table B3, QuadFormer and POMO (8aug) exhibit broadly comparable performance on TSP-50. Of the 1,280 instances, the two methods produce identical tour lengths on 727; QuadFormer achieves a lower gap on 161 instances, whereas POMO does so on 392. Although the Wilcoxon signed-rank test indicates a statistically significant difference ($p = 2.0 \times 10^{-5}$), the difference in mean gap is only -0.02 %p. This discrepancy is largely attributable to the substantially larger gains achieved by QuadFormer on the instances where it performs better. Notably, QuadFormer maintains an optimality gap below 1% on every instance, whereas POMO exceeds this threshold on five. Overall, the two methods are comparable in typical performance, while QuadFormer exhibits slightly stronger worst-case robustness.

On TSP-100 (Table B4) and TSP-500 (Table B5), QuadFormer consistently achieves lower optimality gaps than all baselines. These improvements are statistically significant under the Holm-corrected Wilcoxon signed-rank test, and QuadFormer achieves lower gaps on the majority of instances in each pairwise comparison.

As reported in Table B6, QuadFormer and the model proposed in Luo et al. (2025) exhibit comparable average performance on TSP-1000, with no statistically significant difference under the Wilcoxon signed-rank test ($p = 0.371$). The model proposed in Luo et al. (2025) performs better on a slightly larger number of instances (69 versus

Method	Avg Gap (%)	Max Gap (%)	# Gap > 1%	Win/Tie/Loss	Wilcoxon p
QuadFormer ($m = 4$)	**-0.02**	**0.38**	**0**	–	–
POMO (8aug)	0.00	1.44	5	161/727/392	2.0×10^{-5}
LEHD	0.53	12.88	241	957/266/57	7.5×10^{-148}
Pointerformer	0.10	2.56	28	540/596/144	6.2×10^{-72}
INViT-3V	0.24	1.86	45	988/109/183	2.7×10^{-157}

Table B3: Results of the statistical stability test on 1,280 TSP-50 instances. QuadFormer uses multi-start decoding at inference time. *# Gap > 1%* is the number of instances on which that method's gap exceeds 1%, indicating catastrophic failures.

Method	Avg Gap (%)	Max Gap (%)	# Gap > 1%	Win/Tie/Loss	Wilcoxon p
QuadFormer ($m = 4$)	**-0.01**	**0.55**	**0**	–	–
POMO (8aug)	0.11	2.46	15	726/190/364	2.2×10^{-64}
LEHD	0.51	7.44	228	1053/86/141	7.0×10^{-163}
Pointerformer	0.46	2.63	183	1125/56/99	5.4×10^{-188}
INViT-3V	1.12	3.63	708	1276/1/3	4.5×10^{-210}

Table B4: Results of the statistical stability test on 1,280 TSP-100 instances. QuadFormer uses multi-start decoding at inference time. *# Gap > 1%* is the number of instances on which that method's gap exceeds 1%, indicating catastrophic failures.

Method	Avg Gap (%)	Max Gap (%)	# Gap > 3%	Win/Tie/Loss	Wilcoxon p
QuadFormer ($m = 4$)	**0.85**	**1.62**	**0**	–	–
LEHD	1.30	4.41	2	99/0/29	4.1×10^{-12}
Luo et al. (2025)	1.74	11.21	12	107/0/21	3.2×10^{-16}
Pointerformer	3.64	4.87	117	128/0/0	3.8×10^{-22}
INViT-3V	4.66	5.88	127	128/0/0	3.8×10^{-22}

Table B5: Results of the statistical stability test on TSP-500. QuadFormer uses multi-start decoding at inference time. *# Gap > 3%* is the number of instances on which that method's gap exceeds 3%, indicating catastrophic failures.

59), whereas QuadFormer demonstrates substantially greater worst-case robustness: its maximum optimality gap is only 1.83%, compared with 8.54% for the model proposed in Luo et al. (2025), and it incurs no failures, whereas the model proposed in Luo et al. (2025) fails on 11 instances. These results indicate that, despite similar typical performance, QuadFormer provides more reliable performance in cases. Notably, QuadFormer's optimality gap remains below 3% on every test instance, while each baseline exceeds this threshold on at least one instance.

Method	Avg Gap (%)	Max Gap (%)	# Gap > 3%	Win/Tie/Loss	Wilcoxon p
QuadFormer ($m = 4$)	**1.18**	**1.83**	**0**	–	–
LEHD	2.63	8.47	29	126/0/2	3.8×10^{-22}
Luo et al. (2025)	1.48	8.54	11	59/0/69	0.371
Pointerformer	7.28	8.81	128	128/0/0	3.8×10^{-22}
INViT-3V	5.44	6.36	128	128/0/0	3.8×10^{-22}

Table B6: Results of the statistical stability test on TSP-1000. QuadFormer uses multi-start decoding at inference time. *# Gap > 3%* is the number of instances on which that method's gap exceeds 3%, indicating catastrophic failures.

References

Merz, P., Freisleben, B., *et al.*: Memetic algorithms for the traveling salesman problem. complex Systems **13**(4), 297–346 (2001)

Honda, K., Nagata, Y., Ono, I.: A parallel genetic algorithm with edge assembly crossover for 100,000-city scale tsps. In: 2013 IEEE Congress on Evolutionary Computation, pp. 1278–1285 (2013). IEEE

Seo, D.-I., Moon, B.R.: Voronoi quantizied crossover for traveling salesman problem. In: GECCO, pp. 544–552 (2002)

Applegate, D., Bixby, R., Cook, W., Chvátal, V.: On the solution of traveling salesman problems (1998)

Vaswani, A.: Attention is all you need. Advances in Neural Information Processing Systems (2017)

Kenton, J.D.M.-W.C., Toutanova, L.K.: Bert: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of naacL-HLT, vol. 1 (2019). Minneapolis, Minnesota

Arora, S., Liang, Y., Ma, T.: A simple but tough-to-beat baseline for sentence embeddings. In: International Conference on Learning Representations (2017)

Mei, J., Chen, L.-C., Yuille, A., Xie, C.: Spformer: Enhancing vision transformer with superpixel representation. arXiv preprint arXiv:2401.02931 (2024)

Zhang, Z., Han, X., Liu, Z., Jiang, X., Sun, M., Liu, Q.: Ernie: Enhanced language representation with informative entities. arXiv preprint arXiv:1905.07129 (2019)

Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Žídek, A., Potapenko, A., *et al.*: Highly accurate protein structure prediction with alphafold. nature **596**(7873), 583–589 (2021)

Kool, W., Van Hoof, H., Welling, M.: Attention, learn to solve routing problems! arXiv preprint arXiv:1803.08475 (2018)

Kwon, Y.-D., Choo, J., Kim, B., Yoon, I., Gwon, Y., Min, S.: Pomo: Policy optimization with multiple optima for reinforcement learning. Advances in Neural Information Processing Systems **33**, 21188–21198 (2020)

Ma, Y., Li, J., Cao, Z., Song, W., Zhang, L., Chen, Z., Tang, J.: Learning to iteratively solve routing problems with dual-aspect collaborative transformer. Advances in Neural Information Processing Systems **34**, 11096–11107 (2021)

Jin, Y., Ding, Y., Pan, X., He, K., Zhao, L., Qin, T., Song, L., Bian, J.: Pointerformer: Deep reinforced multi-pointer transformer for the traveling salesman problem. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, pp. 8132–8140 (2023)

Fang, H., Song, Z., Weng, P., Ban, Y.: Invit: A generalizable routing problem solver with invariant nested view transformer. arXiv preprint arXiv:2402.02317 (2024)

Applegate, D.L., Bixby, R.E., Chvatal, V., Cook, W.J.: The Traveling Salesman Problem: A Computational Study (Princeton Series in Applied Mathematics). Princeton University Press (2007)

Croes, G.A.: A method for solving traveling-salesman problems. Operations research **6**(6), 791–812 (1958)

Lin, S.: Computer solutions of the traveling salesman problem. Bell System Technical Journal **44**(10), 2245–2269 (1965)

Helsgaun, K.: An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems. Roskilde: Roskilde University **12**, 966–980 (2017)

Lin, S., Kernighan, B.W.: An effective heuristic algorithm for the traveling-salesman problem. Operations research **21**(2), 498–516 (1973)

Xin, L., Song, W., Cao, Z., Zhang, J.: Neurolkh: Combining deep learning model with lin-kernighan-helsgaun heuristic for solving the traveling salesman problem. Advances in Neural Information Processing Systems **34**, 7472–7483 (2021)

Vinyals, O., Fortunato, M., Jaitly, N.: Pointer networks. Advances in neural information processing systems **28** (2015)

Bresson, X., Laurent, T.: The transformer network for the traveling salesman problem. arXiv preprint arXiv:2103.03012 (2021)

Kim, M., Park, J., Park, J.: Sym-nco: Leveraging symmetricity for neural combinatorial optimization. Advances in Neural Information Processing Systems **35**, 1936–1949 (2022)

Luo, F., Lin, X., Liu, F., Zhang, Q., Wang, Z.: Neural combinatorial optimization with heavy decoder: Toward large scale generalization. Advances in Neural Information Processing Systems **36**, 8845–8864 (2023)

Luo, F., Lin, X., Wu, Y., Wang, Z., Xialiang, T., Yuan, M., Zhang, Q.: Boosting neural combinatorial optimization for large-scale vehicle routing problems. In: The Thirteenth International Conference on Learning Representations (2025)

Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., Zagoruyko, S.: End-to-end object detection with transformers. In: European Conference on Computer Vision, pp. 213–229 (2020). Springer

Ray, P.P.: Chatgpt: A comprehensive review on background, applications, key challenges, bias, ethics, limitations and future scope. Internet of Things and Cyber-Physical Systems **3**, 121–154 (2023)

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al.: Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288 (2023)

Fu, Z.-H., Qiu, K.-B., Zha, H.: Generalize a small pre-trained model to arbitrarily large tsp instances. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol.

35, pp. 7474–7482 (2021)
Sun, Z., Yang, Y.: Difusco: Graph-based diffusion solvers for combinatorial optimization. Advances in Neural Information Processing Systems **36**, 3706–3731 (2023)
Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101 (2017)
Reinelt, G.: Tsplib—a traveling salesman problem library. ORSA journal on computing **3**(4), 376–384 (1991)
Coulom, R.: Efficient selectivity and backup operators in monte-carlo tree search. In: International Conference on Computers and Games, pp. 72–83 (2006). Springer